

CSE481/CSE371 Spring 2026

Introduction to Arduino



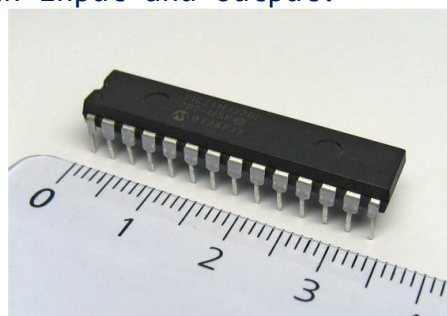
Lab Objectives:

- Understand the fundamentals of microcontrollers and their role in embedded systems.
- Learn the basics of Arduino, including hardware components and development environment.
- Explore different types of Arduino boards and their applications.
- Gain hands-on experience with the Arduino IDE and essential programming concepts.
- Implement basic input and output operations using digital and analog signals.
- Develop simple Arduino-based projects, including LED control and pushbutton interaction.
- Apply control structures and programming logic to automate tasks.
- Introduce concepts of switch debouncing and pull-up/pull-down resistors for stable input handling.

Microcontroller Definition

A microcontroller is a computer. All computers have several things in common:

1. All computers have a CPU (central processing unit) that executes programs.
2. The computer has some RAM (random-access memory) where it can store "variables."
3. And the computer has some input and output devices so it can talk to people. On your desktop machine, the keyboard and mouse are input devices and the monitor and printer are output devices. A hard disk is an I/O device. It handles both input and output.



What is Arduino

- An Arduino board consists of an Atmel 8-bit AVR microcontroller with complementary components that facilitate programming and incorporation into other circuits.
- It's an open-source physical computing platform based on a simple microcontroller board, and a development environment for writing software for the board.



Arduino Family

I. Arduino UNO

- Microcontroller: ATmega328P
- Operating Voltage: 5V
- Digital I/O Pins: 14 (6 PWM)
- Analog Input Pins: 6
- Clock Speed: 16 MHz
- USB Port: Type-B
- Best For: Learning, simple robotics, sensor-based projects



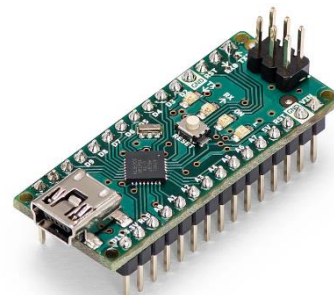
II. Arduino Mega

- Microcontroller: ATmega2560
- Operating Voltage: 5V
- Digital I/O Pins: 54 (15 PWM)
- Analog Input Pins: 16
- Clock Speed: 16 MHz
- USB Port: Type-B
- Best For: Robotics, automation, IoT projects with many sensors



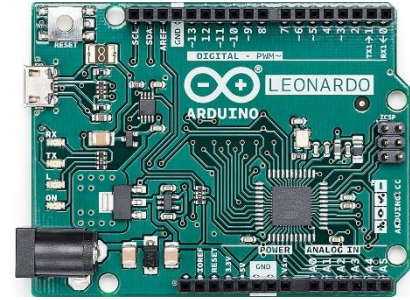
III. Arduino Nano

- Microcontroller: ATmega328P
- Operating Voltage: 5V
- Digital I/O Pins: 14 (6 PWM)
- Analog Input Pins: 8
- Clock Speed: 16 MHz
- USB Port: Mini-USB
- Best For: Wearables, compact projects, portable electronics



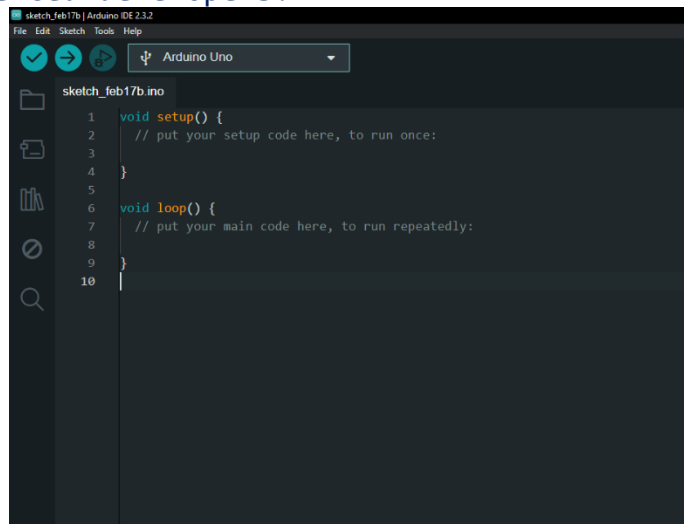
IV. Arduino Leonardo

- Microcontroller: ATmega32U4
- Operating Voltage: 5V
- Digital I/O Pins: 20 (7 PWM)
- Analog Input Pins: 12
- Clock Speed: 16 MHz
- USB Port: Micro-USB
- Best For: Keyboard/mouse emulation, interactive projects



Arduino IDE

The Arduino Integrated Development Environment (IDE) is the software platform used to write, compile, and upload programs (sketches) to Arduino boards. It's designed to be simple and user-friendly, making it an ideal tool for both beginners and experienced developers.



Key Features of the Arduino IDE

1. Cross-Platform Compatibility

- The IDE is compatible with Windows, macOS, and Linux operating systems.

2. Simple Interface

- The IDE is known for its clean and intuitive interface, with two main areas:
 - Editor (where you write your code).
 - Message Area (shows errors and success messages).
 - Toolbar (to upload code, select board, etc.).

3. Code Editor

- The editor supports syntax highlighting, which makes reading and writing code easier.
- It also includes features like auto-indentation and line numbering.

4. Built-in Libraries

- The Arduino IDE comes with a vast library of pre-written functions to simplify tasks, such as controlling motors, sensors, and displays.
- Libraries can be added and managed via the IDE for extra functionality.

5. Serial Monitor

- The Serial Monitor is built into the IDE, which allows you to send and receive data from the Arduino board during runtime.
- You can debug your code by printing information to the Serial Monitor (e.g., sensor values or status messages).

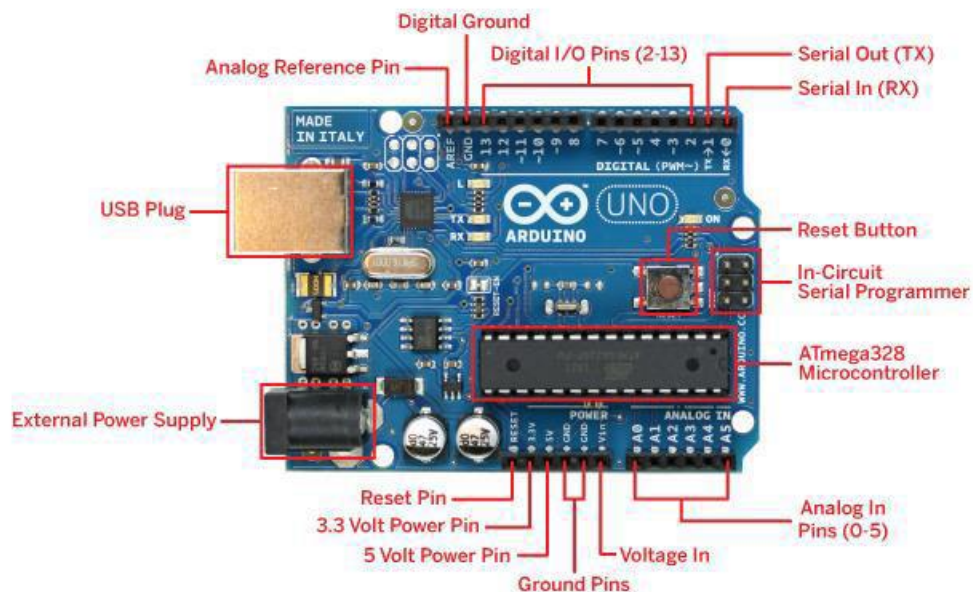
6. Boards and Port Management

- The IDE allows you to select your specific Arduino board model and the appropriate serial port (USB connection to the board).
- Board Managers allow you to add new boards and update existing ones

Arduino UNO Specifications

- Microcontroller: ATmega328P
- Operating Voltage: 5V
- Input Voltage (recommended): 7-12V
- Input Voltage (limit): 6-20V
- Digital I/O Pins: 14 (of which 6 provide PWM output)
- PWM Digital I/O Pins: 6
- Analog Input Pins: 6
- DC Current per I/O Pin: 20 mA
- DC current for 3.3V Pin: 50 mA
- Flash Memory: 32 KB (ATmega328P) of which 0.5 KB used by bootloader
- SRAM: 2 KB (ATmega328P)
- EEPROM: 1 KB (ATmega328P)
- Clock Speed: 16 MHz
- LED_BUILTIN: 13
- Length: 68.6 mm
- Width: 58.4 mm
- Weight: 25 g

Arduino UNO Pins



I. Digital Pins (0 to 13):

- Digital Pins (0 to 13) are general-purpose I/O pins that can be configured as input or output.
- These pins can read digital signals (HIGH or LOW) or output HIGH (5V) or LOW (0V) signals.
- Some of these pins have special functions that include PWM output, serial communication, and interrupts.

Pin Functions:

- **Pin 0 and Pin 1 (RX, TX):** Used for serial communication (RX = Receive, TX = Transmit). These pins are connected to the USB-to-serial converter.
- **Pin 3, 5, 6, 9, 10, and 11 (PWM Pins):** These pins can output Pulse Width Modulation (PWM) signals, which are useful for controlling the brightness of LEDs, speed of motors, etc.
- **Pin 2, 4, 7, 8, 12, and 13:** General-purpose digital I/O pins, can be used for reading or outputting signals.

II. Analog Pins (A0 to A5):

- Analog Pins (A0 to A5) are used to read analog signals. These pins are connected to the ADC (Analog-to-Digital Converter) in the microcontroller.
- Each analog pin can read values in the range of 0 to 1023 (for a 10-bit ADC), corresponding to voltages from 0V to 5V.

Pin Functions:

- These pins can also be used as digital inputs or outputs if needed.

III. Power Pins:

- **5V Pin:** Supplies 5V of power from the Arduino board to external components (e.g., sensors, modules). It is derived from the USB power or the external DC power supply.
- **3.3V Pin:** Provides 3.3V of power (less common, but useful for low-power components).
- **GND Pins (Ground):** There are multiple GND pins that provide a common reference point for the circuits connected to the Arduino. These pins are used to close the circuit by connecting the negative side of your components.
- **Vin Pin:** This pin is used to supply power to the Arduino board when using an external power source (usually 7-12V). It provides voltage to the Arduino's voltage regulator, which regulates the voltage to 5V.

IV. Reset Pin:

- The reset pin is used to restart the Arduino Uno program. It connects to the reset circuitry on the board, which triggers the ATmega328P to reset the program, similar to pressing the reset button on the board.

V. Crystal Oscillator (16 MHz):

- The 16 MHz crystal oscillator is responsible for providing the clock signal to the ATmega328P microcontroller, ensuring that it operates at the correct speed.
- This crystal determines the timing for all operations on the Arduino Uno.

VI. USB-B Port:

- The USB-B port is used to connect the Arduino Uno to a computer via a USB cable.
- This port is used for programming the Arduino and for providing power when the Arduino is connected to the computer.

VII. ICSP (In-Circuit Serial Programming) Header:

- The ICSP header is used to directly program the ATmega328P microcontroller via an external programmer.
- This is typically used when you want to reprogram the microcontroller directly without using the bootloader (e.g., for custom applications).

VIII. AREF Pin (Analog Reference Pin):

- The AREF pin is used to provide a reference voltage for the analog-to-digital converter (ADC).
- By default, the ADC uses the 5V voltage reference, but you can provide a different voltage to the AREF pin if you need more precise analog readings.

IX. Voltage Regulator (LM7805):

- The LM7805 voltage regulator provides 5V power to the board from an external supply (if using a supply voltage of 7V or greater).
- It helps ensure that the Arduino Uno gets a stable 5V supply regardless of the input voltage (within the specified range).

C Review

Datatypes

I. Primitive (Predefined) Datatypes

- **char**: used to store one character in a variable, requires 1 byte of memory space.
- **int**: used to store integer numbers, requires 4 bytes of memory space.
- **float**: used to store single-precision floating numbers, requires 4 bytes of memory space.
- **double**: used to store double-precision floating numbers, requires 8 bytes of memory space.

a. Datatypes Modifiers

- **unsigned**: modifies the datatype to store positive numbers and zero.
- **signed**: modifies the datatype to store negative and positive numbers and zero.
- **short**: modifies only int or char datatypes to be half in size and store less number range.
- **long**: modifies the datatypes to be double in size and store more number range.

b. Datatypes Size

Datatype	Size in bytes	Range of values
bool	1	true (1) / false (0)
char	1	-128 to 127 (signed)
unsigned char	1	0 to 255
int	2	-32,768 to 32,767
unsigned int	2	0 to 65,535
short	2	-32,768 to 32,767
unsigned short	2	0 to 65,535
long	4	-2,147,483,648 to 2,147,483,647
unsigned long	4	0 to 4,294,967,295
float	4	1.2E-38 to 3.4E+38
double	8	1.7E-308 to 1.7E+308

II. Derived Datatypes

a. Arrays

An array is a collection of elements of the same data type stored in contiguous memory locations. It allows you to store multiple values of the same type under a single variable name, accessed via an index.

Key Points:

- Arrays can be of any type, such as int, float, char, etc.
- The size of an array must be specified at the time of declaration.
- Arrays are zero-indexed, meaning the first element is at index 0.

Types of Arrays:

- **One-dimensional array:** `int arr[10];`
- **Two-dimensional array:** `int matrix[3][4];`
- **Multi-dimensional array:** Can be extended to more dimensions like `int cube[3][3][3]`

Example:

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50}; // Declares an array of 5 integers

    // Accessing array elements
    for (int i = 0; i < 5; i++) {
        printf("Element at index %d: %d\n", i, arr[i]);
    }
    return 0;
}
```

b. Pointers

A pointer is a variable that stores the memory address of another variable. Pointers are a powerful feature of C and are used for dynamic memory allocation, arrays, functions, and data structures like linked lists.

Key Points:

- A pointer must be declared before it can be used.
- The * operator is used to declare a pointer, and the & operator is used to obtain the address of a variable.
- The * operator is also used for dereferencing, i.e., accessing the value at the address stored in the pointer.

Example:

```
#include <stdio.h>

int main() {
    int x = 10;
    int *ptr = &x; // Pointer to the variable x

    printf("Value of x: %d\n", x); // Output: 10
    printf("Address of x: %p\n", (void*)&x); // Output: Address of x
    printf("Value stored at ptr: %p\n", (void*)ptr); // Output: Address of x
    printf("Value pointed by ptr: %d\n", *ptr); // Output: 10

    return 0;
}
```

Functions

Functions in C are also derived data types since they encapsulate a sequence of statements to perform a task. Functions are defined with a return type and can accept parameters.

Key Points:

- Functions allow for modular programming and code reusability.
- Functions can return values or void if no value is returned.
- Pointers to functions can be used to call functions dynamically.

Example:

```
#include <stdio.h>

// Function declaration
int add(int a, int b);

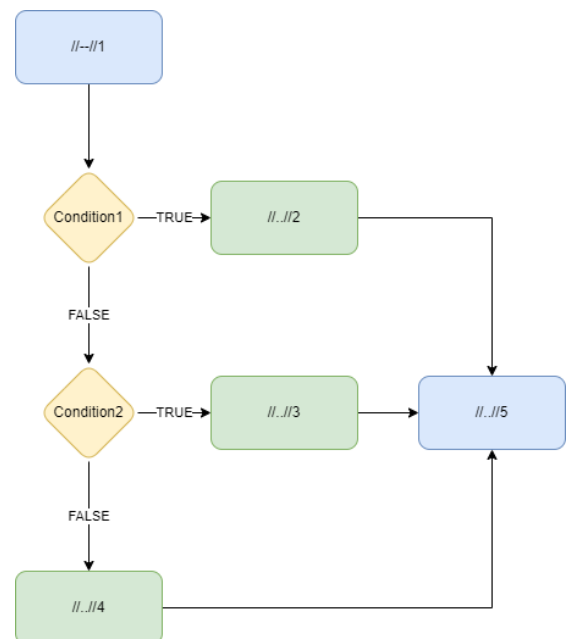
int main() {
    int sum = add(5, 10); // Function call
    printf("Sum: %d\n", sum); // Output: Sum: 15
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}
```

Control Statement

I. If-elseif-else Statement

```
//...//1
if (condition1 )
{
    //...//2
}
else if (condition2)
{
    //...//3
}
else
{
    //...//4
}
//...//5
```



Exercise: What is the output?

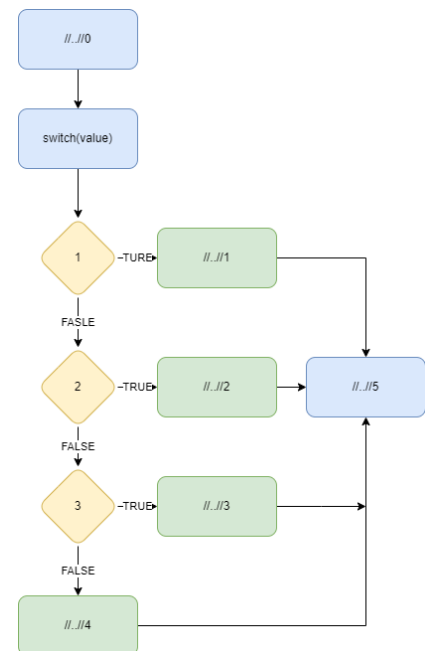
```
#include <stdio.h>

void main() {
    int x = 0;
    printf("Enter x value: ");
    scanf("%d", &x);

    if (x == 5) {
        printf("x = 5\n");
    } else if (x > 5) {
        printf("x > 5\n");
    } else {
        printf("x != 5\n");
    }
}
```

II. switch Statement

```
//...//0
switch (value)
{
case 1:
{ //...//1} break;
case 2:
{ //...//2} break;
case 3:
{ //...//3} break;
default:
{ //...//4} break;
}
//...//5
```



Exercise: What is the output?

```
void main() {
    char traffic_light;

    // Prompt the user to enter a traffic light signal
    printf("Enter traffic light signal (R for Red, G for Green, Y for Yellow): ");
    scanf("%c", &traffic_light);

    // Use switch-case to determine the traffic light action
    switch (traffic_light) {
        case 'R':
        case 'r': // Handles both uppercase and lowercase 'R'
            printf("Stop! The light is red.\n");
            break;

        case 'G':
        case 'g': // Handles both uppercase and lowercase 'G'
            printf("Go! The light is green.\n");
            break;

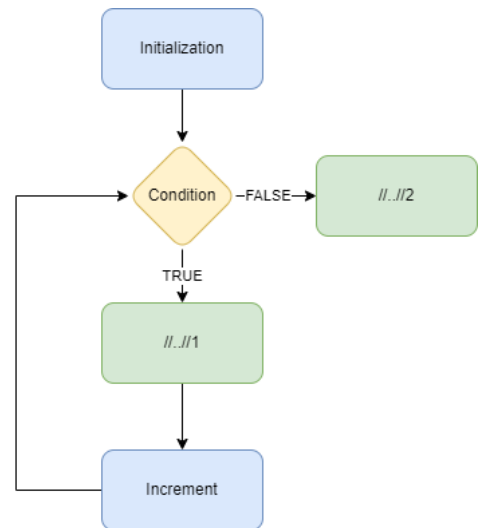
        case 'Y':
        case 'y': // Handles both uppercase and lowercase 'Y'
            printf("Slow down! The light is yellow.\n");
            break;

        default: // Handles invalid inputs
            printf("Invalid input. Please enter R, G, or Y.\n");
            break;
    }
}
```

Loop Statement

I. for Loop

```
for (Initialization; Condition; Increment)
{
    //...//1
}
//...//2
```



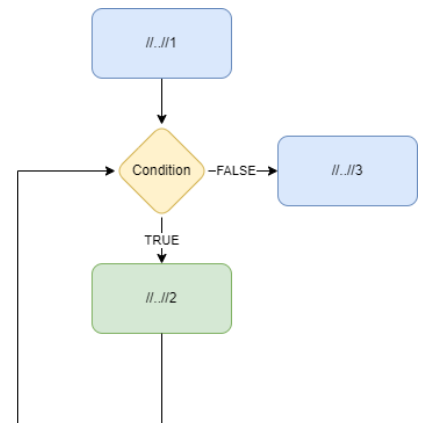
Exercise: What is the output?

```
#include <stdio.h>

int main(void) {
    int i;
    for (i = 1; i <= 10; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

II. while Loop

```
//...//1
while(Condition)
{
    //...//2
}
//...//3
```



Exercise: What is the output?

```
#include <stdio.h>

int main(void) {
    int x, i = 1, z = 0;

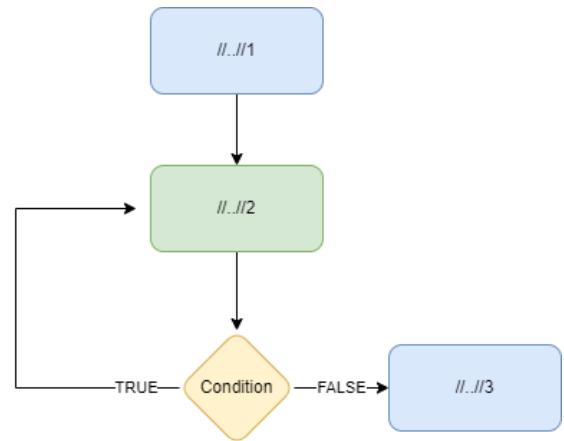
    // Loop to input 5 grades
    while (i <= 5) {
        printf("Enter the grade number %d: ", i);
        scanf("%d", &x);
        z = z + x; // Add the entered grade to the sum
        i++;      // Increment the counter
    }

    // Print the sum of the grades
    printf("Sum = %d\n", z);

    return 0; // Return statement to indicate successful program termination
}
```

III. do..while Loop

```
///  
do{  
///  
}while(Condition);
```



Exercise: What is the output?

```
#include <stdio.h>  
  
int main(void) {  
    int i = 1;  
  
    // Do-while loop to print numbers from 1 to 10  
    do {  
        printf("%d\n", i);  
        i++;  
    } while (i <= 10);  
  
    return 0; // Indicate successful program termination  
}
```

Operators

I. Arithmetic Operators

a. Binary Operators

Symbol	Meaning	Example
+	Addition	result = 10 + 2
-	Subtraction	result = 10 - 2
*	Multiplication	result = 10 * 2
/	Division	result = 10 / 2
%	Reminder	Result = 10 % 2

b. Unary Operators

Symbol	Meaning	Example
x++	Postfix Increment	result = x++
++x	Prefix Increment	result = ++x
x--	Postfix Decrement	result = x--
--x	Prefix Decrement	result = --x

c. Ternary Operators

(condition)? expression : expression

Example

z = (x > y) ? 0 : 1;

II. Bitwise Operators

Symbol	Meaning	Example
&	AND	result = x & y
	OR	result = x y
~	NOT	result = ~x
^	XOR	result = x ^ y

III. Assignment Operators

Symbol	Meaning	Example
=	Assign	x = 2
+=	Addition then Assign	x += 2
-=	Subtraction then Assign	x -= 2
*=	Multiplication then Assign	x *= 2
/=	Division then Assign	x /= 2
&=	AND then Assign	x &= 1
=	OR then Assign	x = 15
^=	XOR then Assign	x ^= 1
<<=	Shift Left then Assign	x <<= 1
>>=	Shift Right then Assign	x >>= 1

Exercise: What is the output?

```
int c = 5;
printf("%d\n", c++);
printf("%d\n", ++c);
c++;
printf("%d\n", c++);
++c;
printf("%d\n", c++);
printf("%d\n", c);
```

Exercise: What is the output?

```
int x = 2, y = 3, z;
z = ((x++) * (x++)) + ((++y) * (++y));
printf("%d\t%d\t%d\n", x, y, z);
```

Arduino Code

Important Arduino Functions

pinMode()

- Arduino function that configures a specific pin as an input or output (pin's mode).

```
pinMode(pinNumber, mode);
```

- **Mode** can be:
 - INPUT
 - OUTPUT

```
pinMode(12, INPUT);  
pinMode(13, OUTPUT);
```

digitalRead()

- This function reads the state of a digital pin (HIGH or LOW).

```
int val = digitalRead(pinNumber);
```

```
int val = digitalRead(12);
```

digitalWrite()

- This function sets a digital pin to either HIGH (5V) or LOW (0V).

```
digitalWrite(pin, value);
```

```
digitalWrite(13, HIGH);  
digitalWrite(13, LOW);
```

analogRead()

- This function reads a continuous range of analog pins. The values are range from 0 to 1023

```
int val = analogRead(pinNumber);
```

```
int val = analogRead(A0);
```

analogWrite()

- The function generates a PWM (analog) signal on certain pins. The signal values range from 0 to 255.

```
analogWrite(pin, value);
```

```
analogWrite(3, 125);
```

delay()

- This function is used for timing and hold the microcontroller for milliseconds

```
delay(TimeVal)
```

```
delay(1000);
```

void setup() vs void loop()

void setup()

- Runs once when the Arduino is powered on or reset.
- Used for initializing settings like pin modes and serial communication.

```
void setup() {  
  // put your setup code here, to run once:  
  pinMode(12, INPUT);  
  pinMode(13, OUTPUT);  
  pinMode(3, OUTPUT);  
}
```

void loop()

- Runs continuously after setup() completes.
- Used to execute repeating tasks such as reading sensors or controlling actuators.

```
void loop() {  
  // put your main code here, to run repeatedly:  
  digitalWrite(13, HIGH);  
  delay(1000);  
  digitalWrite(13, LOW);  
}
```

Inputs VS Outputs

I. Input Components

Inputs are the devices or sensors that provide data to the Arduino from the external environment. These inputs are used to gather information such as temperature, light intensity, motion, or button presses.

Common Input Components:

- **Sensors:** Devices that measure physical quantities and convert them into electrical signals.
Example: A temperature sensor that tells the Arduino the temperature of its surroundings.
- **Switches/Buttons:** Simple devices that can be either ON or OFF.
Example: A push button that when pressed, sends a signal to the Arduino.



- **Potentiometer:** A type of variable resistor that can measure levels of resistance.
Example: It can be used to adjust the brightness of an LED or control the speed of a motor.



II. Output Components

Outputs are the devices or actuators that allow the Arduino to control or influence the environment. These outputs act based on the information received from input sensors or predefined conditions in the program.

Common Output Components:

- **LEDs (Light Emitting Diodes):** A simple way to give visual feedback.
Example: Turning on or off an LED based on certain conditions (e.g., press a button, detect light, etc.).
- **Motors:** Used to produce movement in robotic systems or mechanical parts.
Example: A DC motor that turns a wheel or a servo motor used to position an object.
- **LCD Screens:** Display data or messages visually.
Example: Displaying the temperature reading on an LCD screen.



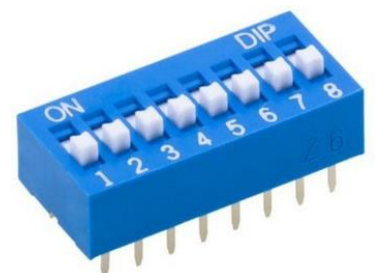
Switch VS Pushbuttons

I. Switches:

A switch is a device that can open or close a circuit, allowing or preventing current from passing through. Switches are typically designed for continuous use and can stay in one position (either on or off) until they are manually moved to the opposite position.

Characteristics of Switches:

- **Types of switches:** There are several types of switches, including toggle switches, slide switches, rocker switches, and rotary switches.
- **Functionality:** A switch can either be in an open state (no current flow) or a closed state (current flows).
- **State Holding:** Once the switch is toggled, it remains in that state (either on or off) until it is manually changed.



II. Pushbuttons:

A pushbutton is a type of switch that momentarily makes or breaks a connection when pressed. When you press the button, it closes the circuit, allowing current to flow, and when you release it, the button opens the circuit, stopping the current. Unlike switches, pushbuttons are not designed to stay in one position. They are typically used in situations where an action is required for a brief moment.

Characteristics of Pushbuttons:

- **Momentary Action:** The button only stays in the pressed state for as long as you are pressing it.
- **Types of Pushbuttons:** There are normally open (NO) and normally closed (NC) pushbuttons. In a normally open button, the circuit is open when the button is not pressed and closes when pressed. A normally closed button does the opposite.
- **State Holding:** A pushbutton does not stay in a particular position. Once released, the button returns to its original state (open or closed).



Pull-Up VS Pull-Down

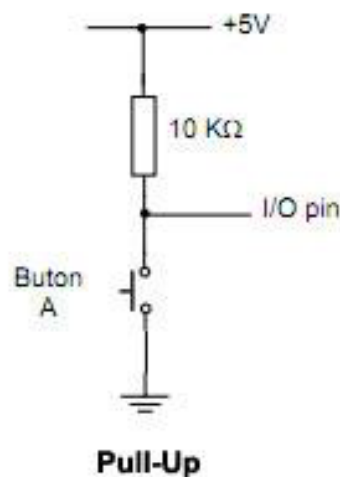
I. Pull-Up Resistor:

A pull-up resistor is connected between the input pin and the positive voltage supply (e.g., +5V or +3.3V). It ensures that the input pin reads HIGH (logic 1) when the switch is open (not pressed).

How It Works:

- When the switch is open (not pressed), the pull-up resistor pulls the input pin to HIGH (connected to the voltage supply).
- When the switch is closed (pressed), the input pin is pulled to LOW (connected to ground, overriding the pull-up resistor).

This setup is often used in cases where the default state should be HIGH (active-high logic) and the input is only driven LOW when a button or switch is pressed.



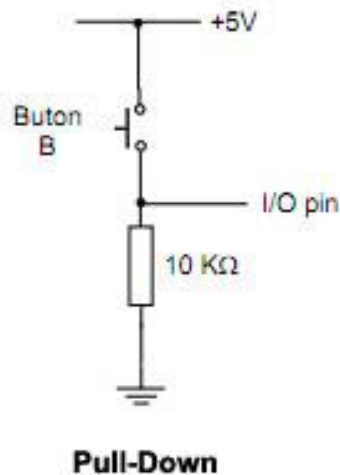
II. Pull-Up Resistor:

A pull-down resistor is connected between the input pin and ground. It ensures that the input pin reads LOW (logic 0) when the switch is open (not pressed).

How It Works:

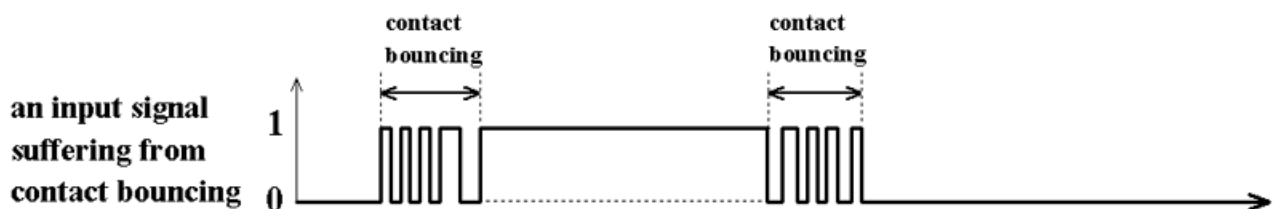
- When the switch is open (not pressed), the pull-down resistor pulls the input pin to LOW (connected to ground).
- When the switch is closed (pressed), the input pin is pulled to HIGH (connected to the positive voltage supply, overriding the pull-down resistor).

This setup is often used when the default state should be LOW (active-low logic), and the input is driven HIGH when the button is pressed.



Switch Bouncing

When a mechanical contact, such as pushbutton, switch, user-interface button, limit switch, relay, or contactor contact, is opened or closed, the contact seldom demonstrates a clean transition from one state to another. When a contact is closed or opened, it will close and open (technically speaking "make" and "break"), many times before finally settling in a stable state.



We can solve this de-bouncing solution by writing code

The first solution:

```
if (btn_reading == HIGH) { //Pull down configuration
    delay(50); // 200 ms delay, increase if the output flickers
    if (btn_reading == HIGH)
    {
        //YOUR CODE
    }
}
```

The second solution using ezbutton library:

```
#include <ezButton.h>

ezButton button (7); // create ezButton object that attach to pin 7;

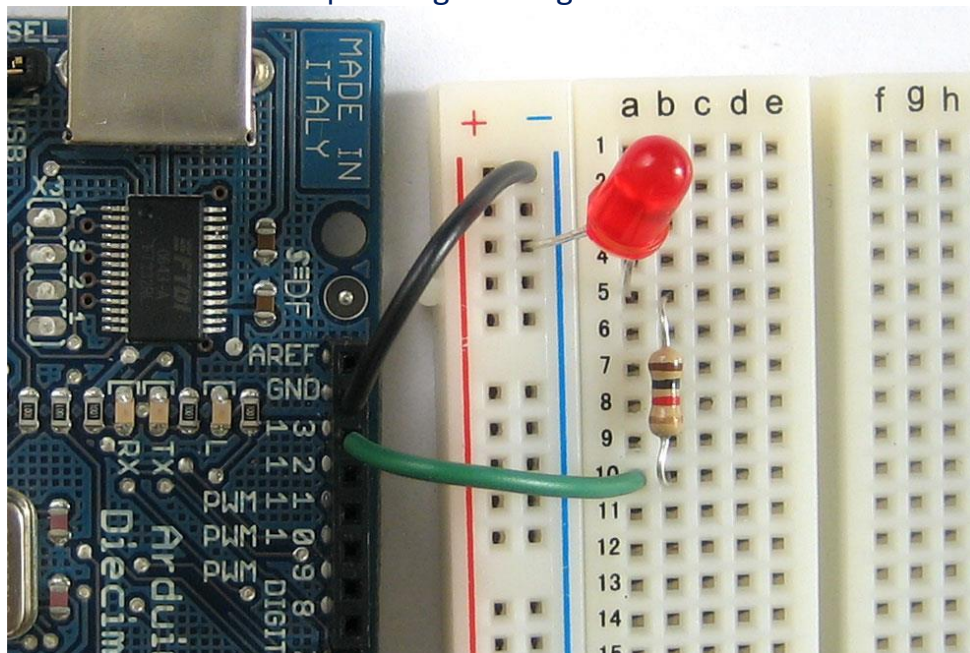
void setup() {
  Serial.begin(9600);
  button.setDebounceTime (50); // set debounce time to 50 milliseconds
}

void loop() {
  button.loop(); // MUST call the loop() function first
  if (button.isPressed())
    Serial.println("The button is pressed");
  if(button.isReleased())
    Serial.println("The button is released");
}
```

Exercises

Exercise 1: Blink using External LED

Connect an LED to any digital pin on your Arduino and put a resistance (270 ohm) in series to limit the current passing through the LED.



Exercise 2: Traffic LEDs

Design a system to demonstrate a street traffic light system.



First state

Duration:6 sec.



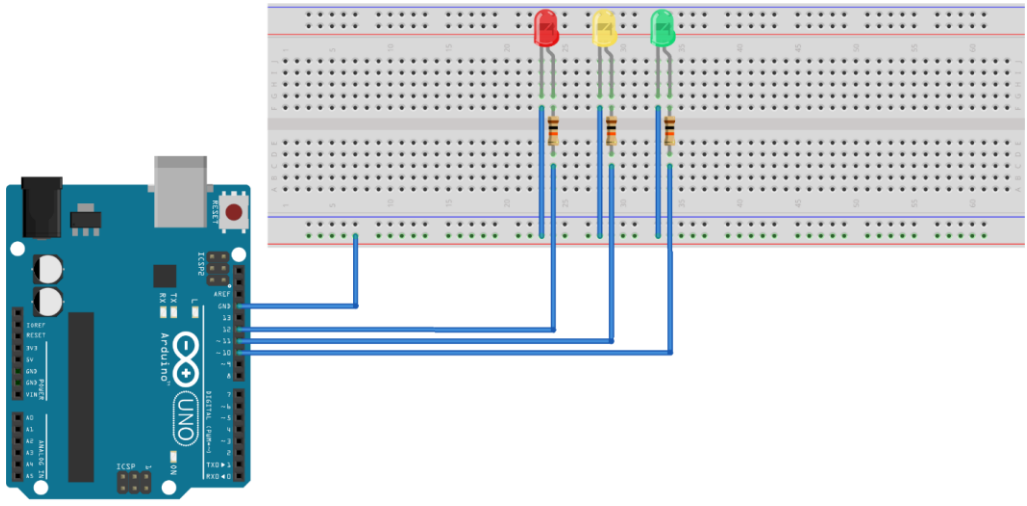
Second state

Duration:4 sec.



Third state

Duration:2 sec.



CSE481/CSE371 Spring 2026

“Sensors”



Lab Objectives:

- I. Learn About Sensor Definition and main working principle
- II. Recap about difference between analog and Digital signals
- III. Explore Arduino's Serial Monitor and Serial Plotter
- IV. Gain Hands-on Experience Using Both analog and Digital Sensors
- V. Introducing voltage divider limitations and alternatives
- VI. Tips on How to choose the right sensor

Sensor Definition & Working Principle

Sensors are essentially devices that detect and respond to environmental changes. These changes can be temperature, light, pressure, or even the presence of chemicals. In essence, electronic sensors act as translators between the physical and the digital world. They convert physical changes into electrical signals, enabling us to measure, analyze, and interact with our surroundings in numerous ways.

A sensor operates through a systematic process that involves several key stages:

Sensor working principle



- **Receptors:** The receptor section senses the input phenomena such as temperature, light, or motion.
- **Transduction:** The input is converted into another form of energy via transduction. For e.g, thermal energy is converted into electrical energy.
- **Signal Conditioning:** The transduced signal is amplified and processed via filtering and digital conversion circuits.
- **Output:** The conditioned signal is converted into a human-readable output format like voltage, current, frequency, digital code, etc.

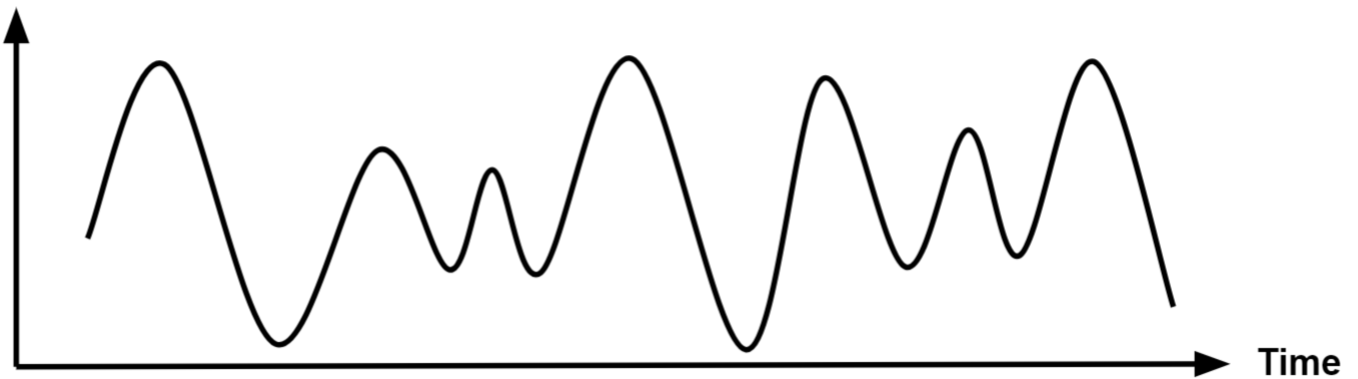
Analog Signals Vs Digital Signals

Analog and digital signals are two fundamental types of electrical signals used to transmit information in various electronic systems. Analog signals are continuous and vary smoothly over time, while digital signals are discrete and represented by a series of discrete values.

Analog signals:

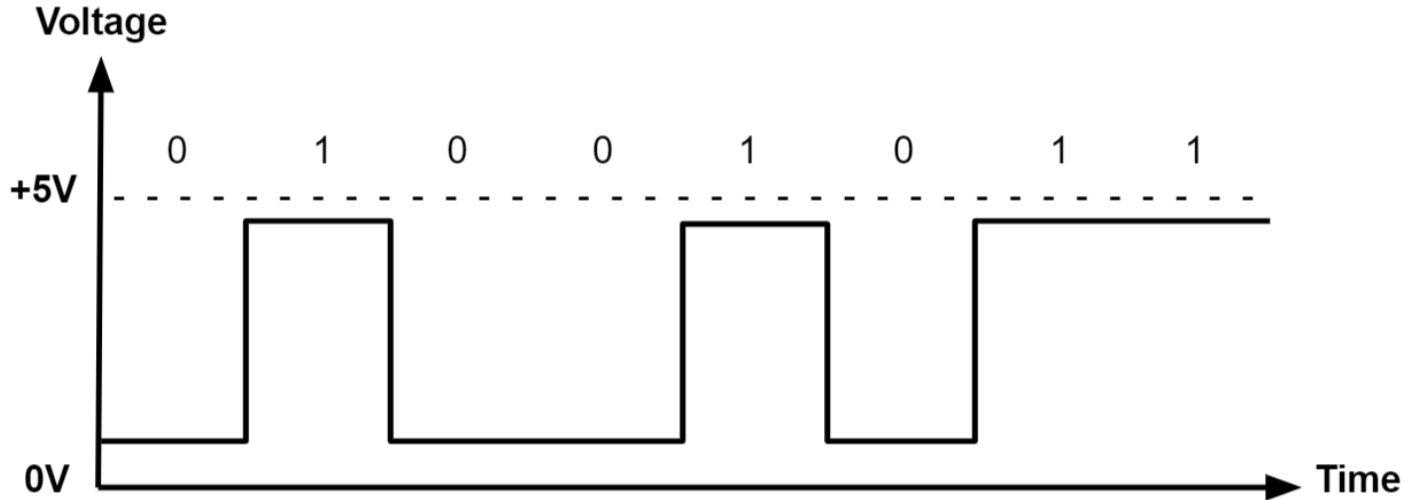
Analog signal is a type of signal that represents continuous data using a continuous range of values. In other words, it can take on any value within a certain range. Analog signals are characterized by their smooth and continuous nature.

Amplitude



Digital Signal

Digital signal is a type of signal that represents data as a sequence of discrete values, typically using binary numbers, it also contains different voltage values. Unlike analog signals, which vary continuously over time, digital signals are discrete and quantified, meaning they only take on specific, distinct values.



Key Differences

	Analog Signal	Digital Signal
Signal Type	Continuous waveforms	Discrete Signals
Processing	Requires complex processing for manipulation.	It is easier to process and manipulate digitally.
Storage	Less efficient for storage due to continuous nature.	More efficient for storage due to discrete values.
Errors	Susceptible to noise and distortion	More resistant to noise and distortion

Analog Vs Digital Sensors:

In a nutshell, one of the key differences between analog and digital sensors is their output resolution. For example, analog sensors provide infinite resolution since they provide a continuous range of values, while digital sensors have a finite resolution determined by the number of bits used to represent the signal. Another difference is the susceptibility to noise. Analog signals are more prone to noise interference since they are continuous and can be affected by external factors such as electromagnetic interference. On the other hand, digital signals are less susceptible to noise since they are discrete and can be easily filtered and processed.

Arduino Serial Monitor

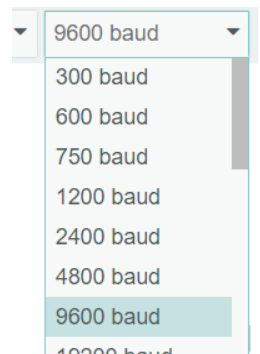
The Serial Monitor is a separate pop-up window that acts as a separate terminal that communicates by receiving and sending Serial Data. See the icon on the far right of the image above.

Initializing Serial Communication

Function: `Serial.begin(baudrate)`

Baud Rate: The number of bits per second (bps) for data transmission. allow faster data transfer but might be more prone to errors if the connection isn't stable.

Note: Make Sure to use the baud rates specified in the Arduino IDE



Printing on The Serial Monitor

- 1) **`Serial.print()`:** Prints data to the serial port, without adding a newline at the end.

```
1 void setup() {
2   // put your setup code here, to run once:
3   Serial.begin(9600);
4   Serial.print("Hello ");
5   Serial.print("World");
6
7
8 }
9
10 void loop() {
11   // put your main code here, to run repeatedly:
12 }
13
```

Serial Monitor × Output

Message (Enter to send message to 'Arduino Uno' on 'COM4')

Hello World

- 2) **`Serial.println()`:** Prints data to the serial port with a newline character at the end, moving the cursor to the next line.

```
1 void setup() {
2   // put your setup code here, to run once:
3   Serial.begin(9600);
4   Serial.println("Hello ");
5   Serial.println("World");
6
7
8 }
9
10 void loop() {
11   // put your main code here, to run repeatedly:
12 }
13
```

Serial Monitor × Output

Message (Enter to send message to 'Arduino Uno' on 'COM4')

Hello
World

Data Type Formatting

```
1 void setup() {
2   // put your setup code here, to run once:
3   Serial.begin(9600);
4   int number = 745;
5   Serial.println(number, DEC); // Decimal Representation
6   Serial.println(number, HEX); // Hexadecimal Representation
7   Serial.println(number, BIN); // Binary Representation
8   Serial.println(number, OCT); // Octal Representation
9   Serial.println(2.1245689, 2); // Floating-point with 3 decimal places
10 }
11
12
13 void loop() {
```

Serial Monitor x Output

Message (Enter to send message to 'Arduino Uno' on 'COM4')

```
745
2E9
1011101001
1351
2.12
```

Printing Data in CSV Format

```
3 void setup() {
4   Serial.begin(9600);
5 }
6
7 void loop() {
8   random_variable = random(0, 1000);
9
10  Serial.print("Variable_1:");
11  Serial.print(random_variable);
12  Serial.print(",");
13  Serial.print("Variable_2:");
14  Serial.println(static_variable);
15  delay(500);
16 }
17
18
```

Serial Monitor x Output

Message (Enter to send message to 'Arduino Uno' on 'COM4')

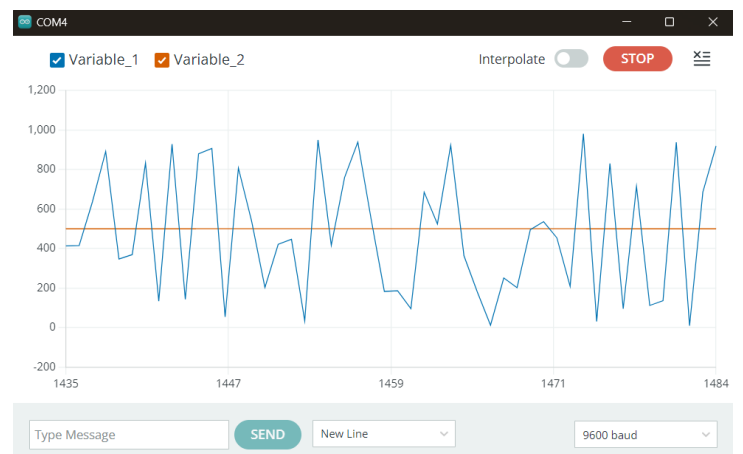
```
Variable_1:878,Variable_2:500
Variable_1:923,Variable_2:500
Variable_1:709,Variable_2:500
```

Serial Plotter

Serial Plotter is a powerful and versatile visualization tool integrated into the Arduino IDE. It allows users to graphically track and display real-time data sent from an Arduino board via the serial interface. While it shares similarities with the Serial Monitor, which outputs data in a text-based, terminal-style format, the Serial Plotter takes data visualization a step further by providing dynamic, interactive graph

Ex 1

```
1 int random_variable;
2 int static_variable = 500;
3
4 void setup() {
5   Serial.begin(9600);
6 }
7
8 void loop() {
9   random_variable = random(0, 1000);
10
11  Serial.print("Variable_1:");
12  Serial.print(random_variable);
13  Serial.print(",");
14  Serial.print("Variable_2:");
15  Serial.println(static_variable);
16 }
17
```



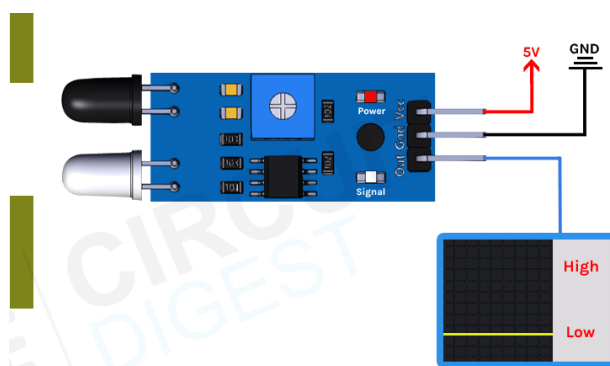
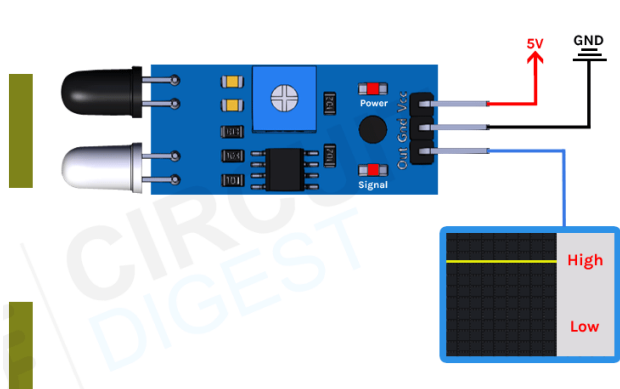
Digital Sensors

IR Sensor

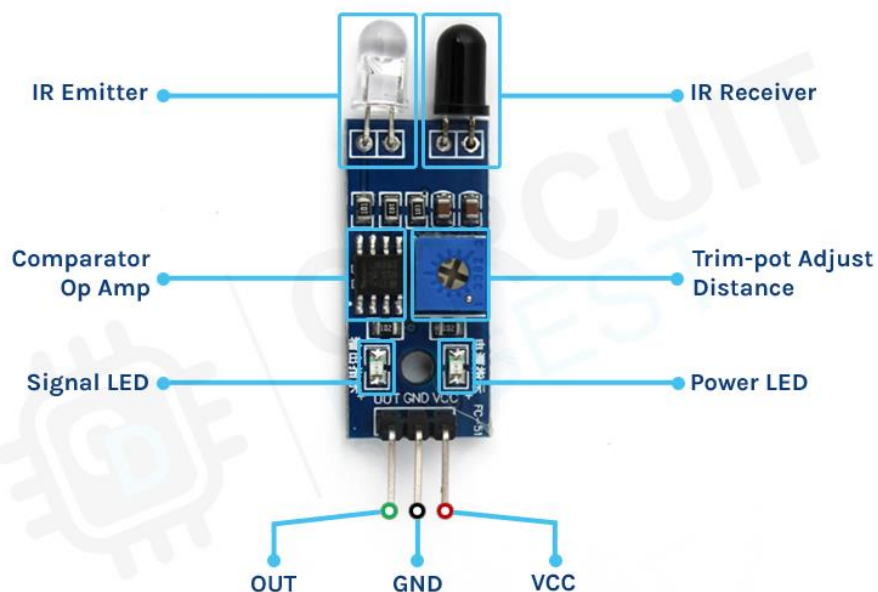
Working Principle

It consists of two main components: the first is the IR transmitter section and the second is the IR receiver section. In the transmitter section, IR led is used and in the receiver section, a photodiode is used to receive an infrared signal.

An IR proximity sensor works by applying a voltage to the onboard Infrared Light Emitting Diode which in turn emits infrared light. This light propagates through the air and hits an object, after that the light gets reflected in the photodiode sensor. If the object is close, the reflected light will be stronger; if the object is far away, the reflected light will be weaker. If you look closely toward the module. When the sensor becomes active it sends a corresponding Low signal through the output pin



IR Parts & Pinouts



IR Specs

Detection Range	between 10 cm and 80 cm
Operating Voltage	3.3V to 5V
Current Consumption	Low current draw, less than 20mA
Wavelength	Around 850 nm to 950 nm
Operating Temperature	Between -10°C to 60°C
Sensing Angle	Between 10 to 45 degrees

Simple IR Code

```
#define IRSensor 9
#define LED 13

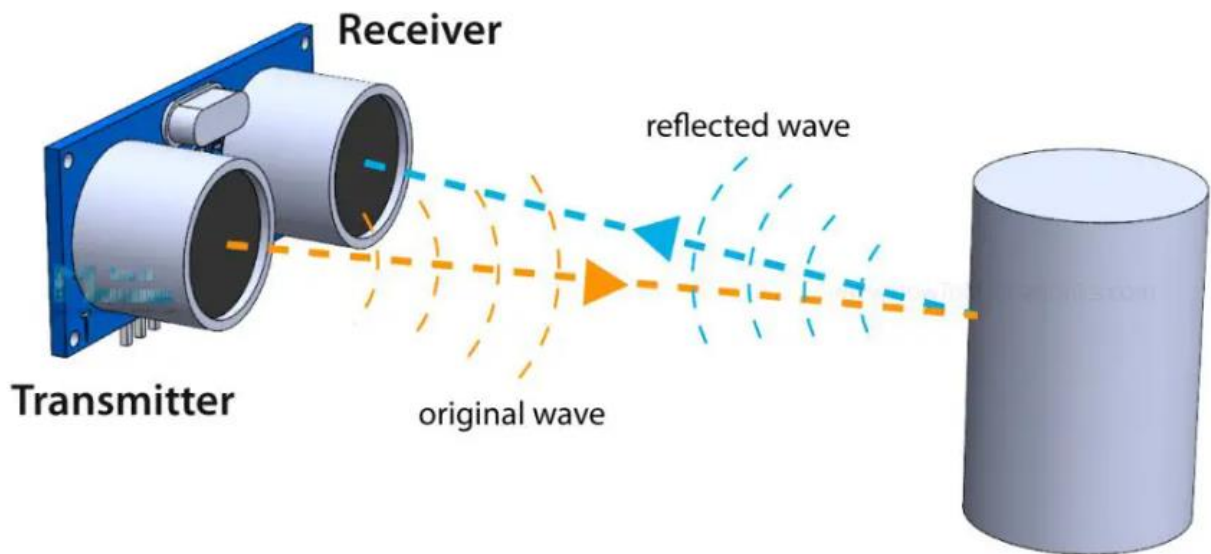
void setup(){
  Serial.begin(9600); // Init Serial at 9600 Baud Rate.
  Serial.println("Serial Working"); // Test to check if serial is working or not
  pinMode(IRSensor, INPUT); // IR Sensor pin INPUT
  pinMode(LED, OUTPUT); // LED Pin Output
}

void loop(){
  int sensorStatus = digitalRead(IRSensor);
  if (sensorStatus == 1) // Check if the pin high or not
  {
    digitalWrite(LED, LOW); // LED LOW
    Serial.println("Motion Detected!"); // print Motion Detected!
  }
  else {
    digitalWrite(LED, HIGH); // LED High
    Serial.println("Motion Ended!"); // print Motion Ended!
  }
}
```

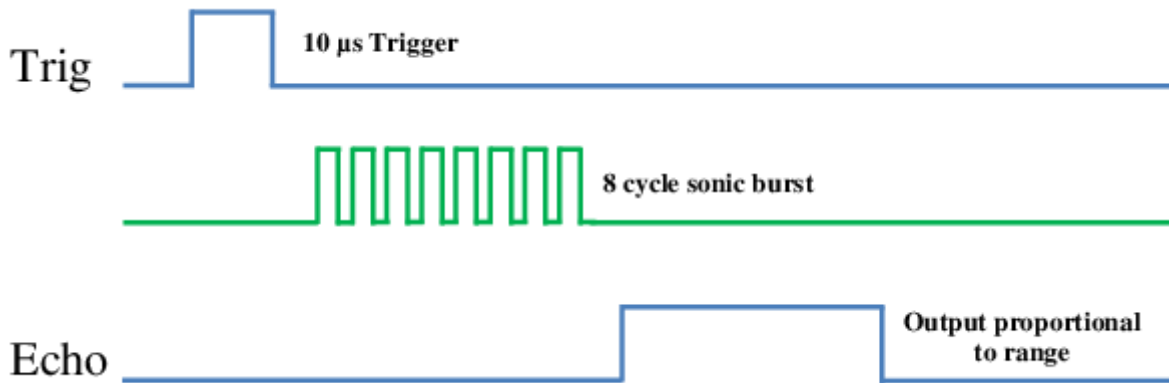
Ultrasonic Sensor

Working Principle

It emits an ultrasound at 40 000 Hz which travels through the air and if there is an object or obstacle on its path It will bounce back to the module. Considering the travel time and the speed of the sound you can calculate the distance.

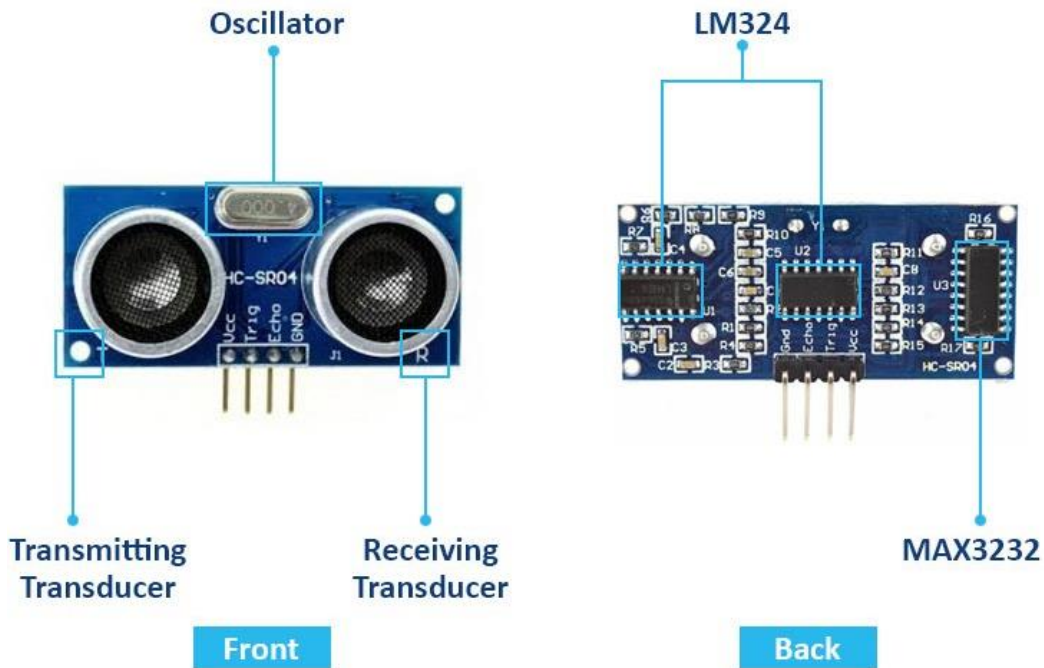


To generate the ultrasound, the Trig pin is to be set on a High State for 10 μ s. That will send out an 8-cycle ultrasonic burst which will travel at the speed of sound. The Echo pins go high right away after that 8-cycle ultrasonic burst is sent, and it starts listening or waiting for that wave to be reflected from an object. If there is no object or reflected pulse, the Echo pin will time-out after 38ms and get back to low state.



Components

An ultrasonic sensor comprises several essential components that work together to measure distances or detect objects using high-frequency sound waves. At its core is the transducer, typically made of a piezoelectric crystal, which both emits and receives ultrasonic waves.



The sensor's operation begins with the ultrasonic transmitter, which sends out bursts of high-frequency sound waves into the environment. On the back of the Ultrasonic sensor, the MAX232 sensor on right side, which converts the electrical signal received from the trig pin. Converts it into ultrasonic pulse and send it's through the transmitter side.

These waves bounce off objects in their path and are detected by the ultrasonic receiver. The two LM324 ICs on the left side take this ultrasonic pulse converts it into electrical signal and sends it to the echo pin. The receiver converts the reflected waves into electrical signals.

Distance Calculation

The time it takes for the sound waves to travel to the object and back is precisely measured by signal processing circuitry within the sensor. This time measurement is crucial for calculating distances and determining the presence of objects in proximity to the sensor.

The Formula of calculating the distance is derived from the following equation:

$$Distance = Speed \times Time$$

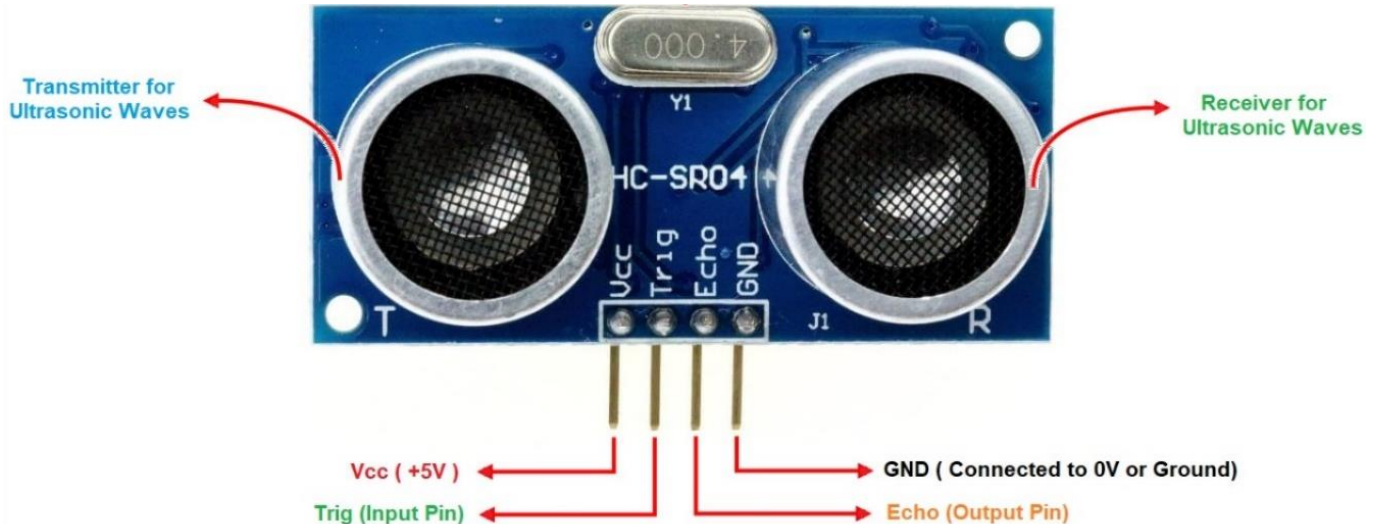
Where *The Speed of Sound* = 343.2 m/s = 0.03432 cm/ μ s

Note: the calculated value represents twice the distance, as the measured time includes the total duration for the sound wave to travel to the target and return to the transducer.

The final formula for calculating the distance can be expressed as following:

$$Distance = \frac{0.03432 * Time(\mu s)}{2}$$

Ultrasonic Sensor Pinout



HC-SR04 Ultrasonic Sensor

Operating Voltage	5V DC
Operating Current	15 mA
Operating Frequency	40KHz
Range	2cm to 400cm
Accuracy	$\pm 3mm$

Example Code

```
// defines pins numbers
#define trigPin 9
#define echoPin 10
// defines variables
long duration;
int distance;
void setup() {
    pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
    pinMode(echoPin, INPUT); // Sets the echoPin as an Input
    Serial.begin(9600); // Starts the serial communication
}
void loop() {
    // Clears the trigPin
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    // Sets the trigPin on HIGH state for 10 micro seconds
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    // Reads the echoPin, returns the sound wave travel time in microseconds
    duration = pulseIn(echoPin, HIGH);
    // Calculating the distance
    distance = duration * 0.03432 / 2;
    // Prints the distance on the Serial Monitor
    Serial.print("Distance: ");
    Serial.println(distance);
}
```

The **pulseIn()** function is utilized to measure the travel time of the ultrasonic pulse, with the resulting value stored in the variable `duration`. This function accepts two arguments: the first specifies the Echo pin being monitored, while the second defines the pulse state to be measured—either HIGH or LOW.

Encoders

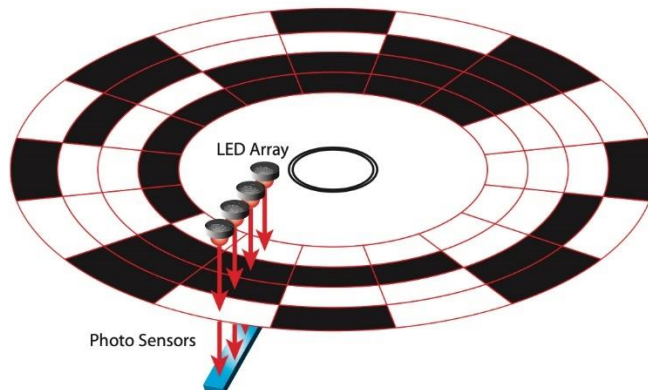
Definition

Encoders are electronic devices that are used to monitor the position and speed of a motor or any rotating mechanism. They play an important role in many industrial and automated systems, as they provide precise and accurate information about the motion and position of a motor.

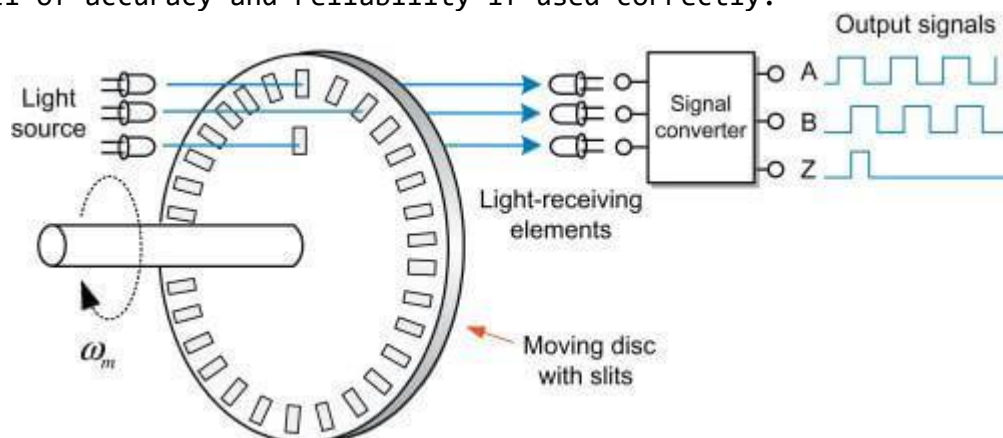
Types

There are two main types of motor encoders: absolute encoders and incremental encoders.

- **Absolute encoders:** provide a unique digital code for each position of the motor shaft, allowing the system to determine the exact position of the motor at any given time. They are commonly used in applications where precise position control is required, such as in robotic systems, motion control systems, and automation systems. They provide a higher level of accuracy and reliability compared to incremental encoders, as they are not affected by power failure or system restart.



- **Incremental encoders:** generate pulses or signals that are proportional to the motion of the motor. These pulses are used to determine the speed and direction of the motor. They are widely used in applications where speed and direction monitoring are required, such as in conveyor systems, machine tools, and material handling systems. They are cost-effective and easy to install and can provide a high level of accuracy and reliability if used correctly.



In This lab will be working with an **incremental encoder**

Code (Measuring Speed and Position)

```
#define ENCODER_PIN_A 2    // Encoder output A
#define ENCODER_PIN_B 3    // Encoder output B

long encoderCount = 0;      // Encoder pulses in the current interval
int currentSpeed = 0;
unsigned long lastTime = 0;
unsigned long currentTime = 0;
int deltaTime = 0;
int currentPosition = 0;
int lastPosition = 0;
int deltaPosition = 0;
// Interrupt to count encoder pulses
void encoderISR() {
    if (digitalRead(ENCODER_PIN_A) == digitalRead(ENCODER_PIN_B)) {
        encoderCount++;
    } else {
        encoderCount--;
    }
}

void setup() {
    pinMode(ENCODER_PIN_A, INPUT_PULLUP);
    pinMode(ENCODER_PIN_B, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(ENCODER_PIN_A), encoderISR, RISING);
    Serial.begin(9600);
}

void loop() {
    currentTime = millis();
    deltaTime = currentTime - lastTime;
    currentPosition = encoderCount;
    deltaPosition = currentPosition - lastPosition;
    currentSpeed = (deltaPosition * 1000L) / deltaTime; // Speed in counts per second
    Serial.print("Current Speed = ");
    Serial.print(currentSpeed);
    Serial.print(" | Encoder Count = ");
    Serial.println(encoderCount);
    lastPosition = currentPosition;
    lastTime = currentTime;
    delay(100);
}
```

attachInterrupt() Arguments:

- **interrupt:** the number of the interrupt (`int`)
- **pin:** the pin number
- **ISR:** the ISR to call when the interrupt occurs; this function must take no parameters and return nothing. This function is sometimes referred to as an interrupt service routine.
- **mode:** defines when the interrupt should be triggered. Four constants are predefined as valid values:
 - **LOW** to trigger the interrupt whenever the pin is low,
 - **CHANGE** to trigger the interrupt whenever the pin changes value
 - **RISING** to trigger when the pin goes from low to high,
 - **FALLING** for when the pin goes from high to low.

Increasing Encoder Resolution via Quadrature Encoder

```
#include <QuadratureEncoder.h>

Encoders Encoder(2,3); // Create an Encoder object name leftEncoder, using digitalpin 2 & 3

void setup() {
  Serial.begin(9600);
}

void loop() {

  long EncoderCount = Encoder.getEncoderCount();
  Serial.println(EncoderCount);
}
```

Reading Analog Data on Arduino

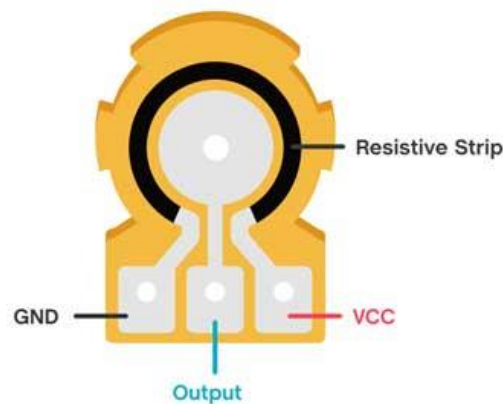
Arduino boards contain a multichannel, 10-bit analog to digital converter. This means that it will map input voltages between 0 and the operating voltage (5V or 3.3V) into integer values between 0 and 1023. On an Arduino UNO, for example, this yields a resolution between readings of 5 volts / 1024 units or, 0.0049 volts (4.9 mV) per unit. See the table below for the usable pins, operating voltage and maximum resolution for some Arduino boards.

Analog Sensors

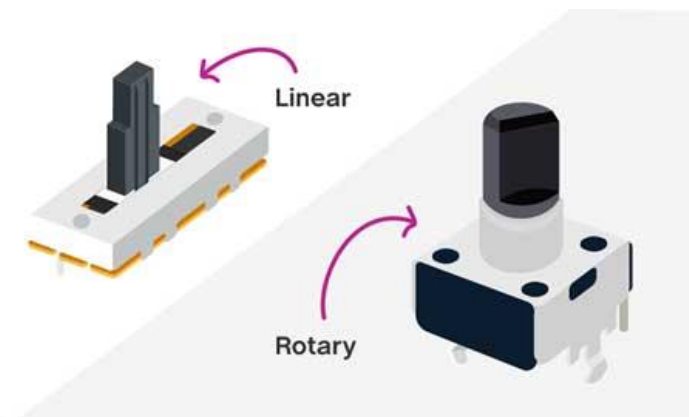
Potentiometers

Basics

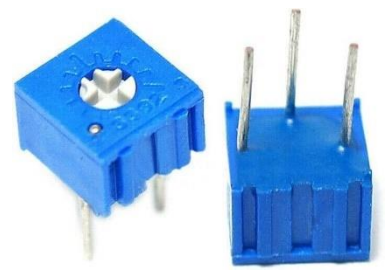
Potentiometers serve as resistor-based components with a mechanical adjustment mechanism that allows manual alteration of their resistance. In contrast to fixed resistors that maintain a constant resistance value, potentiometers act as variable resistors.



Types



Rotary potentiometers utilize angular movement facilitated by a rotating knob and shaft that connect to a wiper element. This



wiper element slides along the resistive element, allowing for changes in resistance and subsequent adjustments to the output. By turning the shaft, the resistance and output can be varied accordingly. Additionally, there are shaftless potentiometers where the wiper is moved using an external tool such as a screwdriver, eliminating the need for a physical shaft. These are commonly referred to as **trimmer potentiometers or trimmers**.

On the other hand, linear potentiometers employ linear or straight-line motion through a sliding mechanism to establish contact with the resistive element. This linear motion enables the variation of resistance and subsequent adjustments to the output.

Parameters to consider Upon Selecting a Potentiometer

- **Resistance:** represents the resistance across the entire length of the resistive element, from one terminal to the other.” Rated Resistance”
- **Rated Power:** the maximum power rating indicates the amount of power a potentiometer can handle to prevent overheating or failure.
- **Resolution:** refers to the accuracy of the potentiometer in terms of its incremental changes in resistance. It is typically expressed as a percentage of the total resistance and represents the **smallest detectable change** in resistance per incremental movement of the wiper on the resistive element.
- **Sliding Noise:** the electronic noise generated during the movement of the potentiometer's internal contact parts. This noise can interfere with the accuracy of the desired signal or introduce unwanted artifacts.
- **Temperature Coefficient:** indicates how the resistance of the potentiometer may change with variations in operating temperature. Potentiometers with lower temperature coefficients offer more stable resistance values over a wide temperature range, ensuring consistent performance in different thermal conditions.
- **Mechanical Life:** the expected lifespan of the potentiometer in terms of the number of cycles it can endure while maintaining its performance specifications.

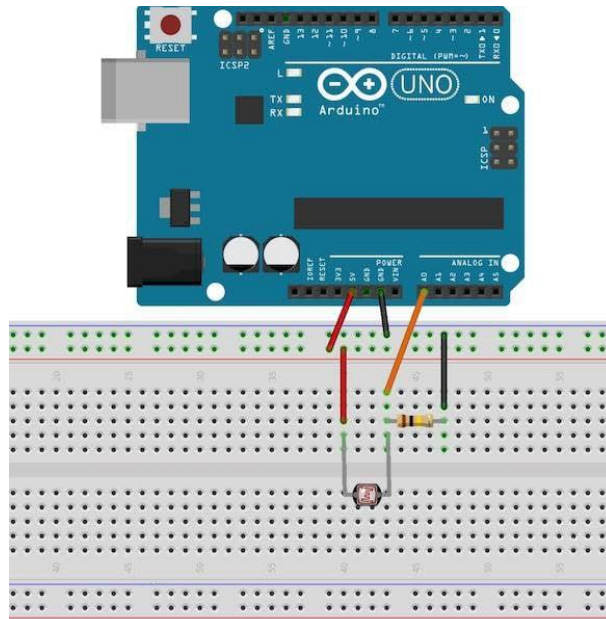
LDR (Light Depending Resistor)

Working Principle

Photoresistors are typically made of semiconductor materials such as cadmium sulfide (CdS) or lead selenide (PbSe). After receiving photon energy, the electrons of these materials transition to the conduction band and become free electrons, thereby increasing the conductivity of the material. Therefore, **the resistance value of LDR is inversely proportional to the intensity of light**, and the stronger the light, the smaller the resistance, The weaker the light, the greater the resistance.

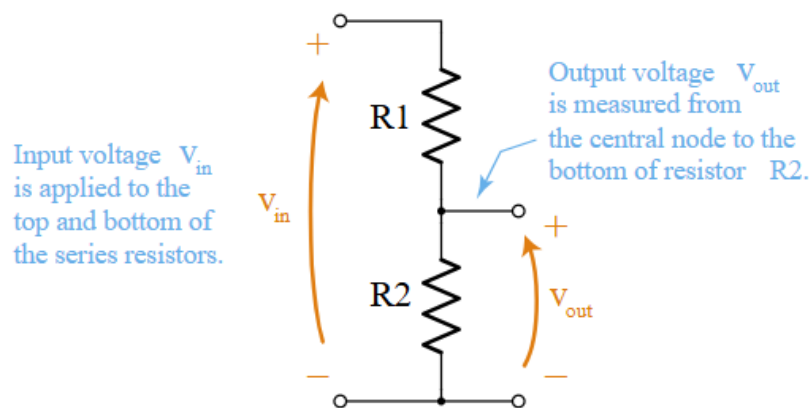
The response speed of LDR is usually slow, especially in situations where light changes rapidly, which needs to be considered when designing circuits. In addition, the **linearity of LDR is poor**, and its resistance change curve is not completely linear in different light intensity ranges.

Sensor Connection



Recap on Voltage Divider

Voltage Divider circuits are used to produce different voltage levels from a common voltage source, but the current is the same for all components in a series circuit.



$$V_{out} = V_{in} * \frac{R_2}{R_1 + R_2}$$

- LDR as R_1 : output voltage increase as the light intensity decreases the LDR should be placed as R_1 as the resistance of an LDR **decreases** when light intensity **increases** and **increases** when light intensity **decreases**.
- LDR as R_2 : output voltage to increase with more light, place the LDR as R_2 . In this case, bright light reduces R_2 , increasing V_o

Potentiometer and LDR Codes

Both the potentiometer and the LDR can be tested using the same code with involves reading an Analog signal

```

#define analogInPin A0 // Analog input pin that the potentiometer is attached to
#define analogOutPin 9 // Analog output pin that the LED is attached to

int sensorValue = 0; // value read from the pot
int outputValue = 0; // value output to the PWM (analog out)

void setup() {
  // initialize serial communications at 9600 bps:
  Serial.begin(9600);
}

void loop() {
  // read the analog in value:
  sensorValue = analogRead(analogInPin);
  // map it to the range of the analog out:
  outputValue = map(sensorValue, 0, 1023, 0, 255);
  // print the results to the Serial Monitor:
  Serial.print("sensor = ");
  Serial.print(sensorValue);
  Serial.print("\t output = ");
  Serial.println(outputValue);
  // wait 2 milliseconds before the next loop for the analog-to-digital
  // converter to settle after the last reading:
  delay(2);
}

```

map(long x, long in_min, long in_max, long out_min, long out_max): it is a function used to re-map a number from one range to another. Its arguments are as follows:

- **x:** Value to be mapped
- **in_min:** The lower bound of the input range.
- **in_max:** The upper bound of the input range.
- **out_min:** The lower bound of the output range.
- **out_max:** The upper bound of the output range.

Changing ADC Reference

Reasons:

- **Improving Measurement Resolution for Low-Voltage Signals:** if the output value of a sensor is 1V then a lot of resolution is lost
- **Increasing Accuracy and Reducing Noise**
- **Handling Different Signals with different ranges**

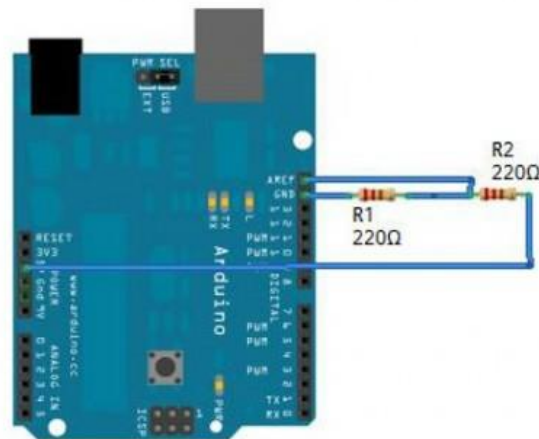
Methodology

For Arduino Uno, you can utilize the Arduino function: **analogReference(type)**

Where the type can be:

- 1) **DEFAULT:** the default analog reference of 5 volts
- 2) **INTERNAL:** a built-in reference, equal to 1.1 volts on the ATmega168 or ATmega328P and 2.56 volts on the ATmega32U4 and ATmega8
- 3) **EXTERNAL:** the voltage applied to the AREF pin (**0 to 5V only**) is used as the reference.

AREF Connection Example



Voltage Divider Limitations and alternatives

Limitations

- Loading effect on the voltage divider when the ADC's input impedance is not much higher than the circuit's output impedance, resulting in distorted values.
- The voltage divider circuit's simplicity limits its ability to address impedance issues and offers little flexibility for altering gain or offset

Alternative (Voltage Follower Circuit)

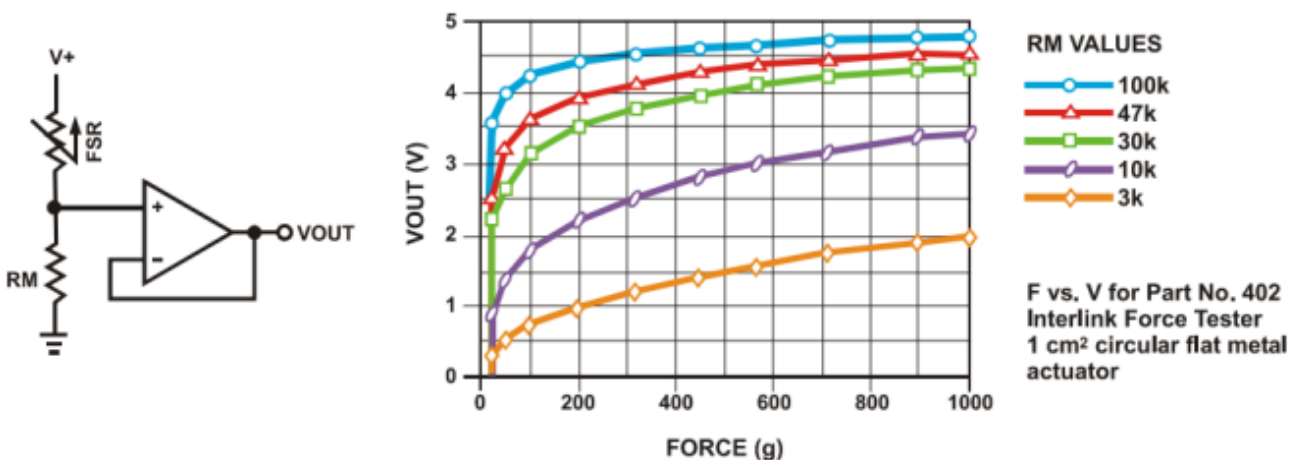
Note: There are other approaches that can be utilized we will cover only one you can search for others if interested

1. Advantages:

- OpAmps have an extremely high input impedance (almost infinite), which considerably reduces the loading effect
- OpAmp acts as a buffer, preserving the integrity of the analog signal and allowing for more exact voltage readings unaffected by the ADC's input impedance

2. Limitation: The offset voltage of the OpAmp might cause a persistent inaccuracy in the output signal, potentially affecting the accuracy of force measurements. However, such mistakes may be controlled using calibration processes. Calibration is an important stage in the deployment of this circuit since it allows for the reduction of offset errors through precise modifications.

Voltage Follower Example using FSR (Force Dependent Resistor)



Tips Sensor Selection

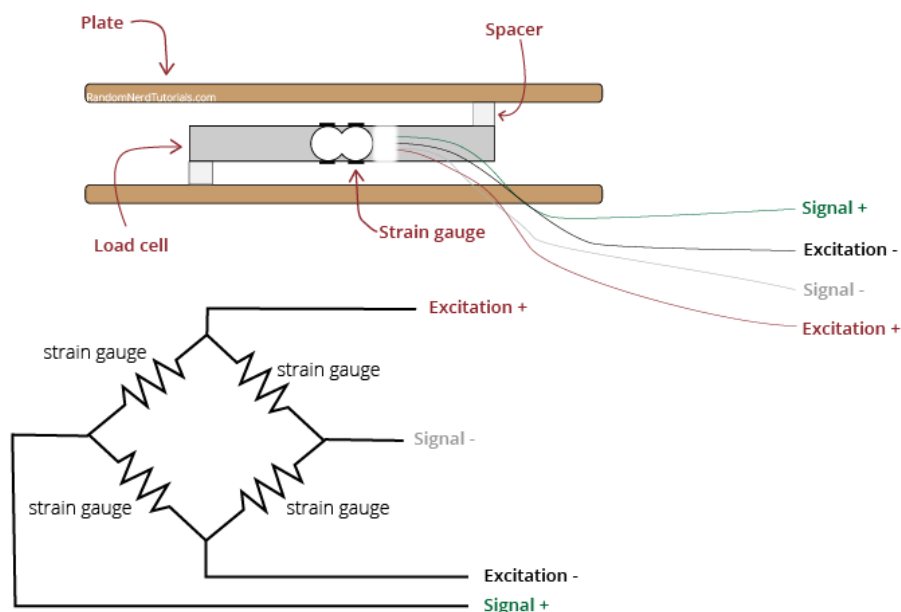
- i. **Accuracy and repeatability:** The sensor's ability to maintain accuracy over time and under varying conditions
- ii. **Sensitivity:** the sensor's ability to detect small changes
- iii. **Compatibility:** it is the question of whether the sensor can be integrated with the current existing system or not
- iv. **Size and Mounting:** The sensor's size will impact the space available for other components in the full assembly. A compact, lightweight sensor is often necessary in constrained spaces or in applications where the sensor must be integrated into existing structures without significant
- v. **Energy & Voltage:** The sensor's electrical requirements are crucial for effective operation and must match your power supply capabilities.
- vi. **Response Speed:** There are three response types to be considered.
 - a. **Time to first reading:** How quickly sensors can give initial data after activation or change in conditions
 - b. **Recovery time after a measurement:** How long until sensor can accurately collect data again
 - c. **Time to stabilize on an accurate reading:** Time needed for output to reflect an accurate measurement
- vii. **Linearity:** refers to how well the output signal changes proportionally to the input physical parameter. High-linearity sensors make data interpretation and system calibration easier since the output signal directly correlates to the measured parameter without needing complex conversions.
- viii. **Durability**
- ix. **Cost**

Unconventional Sensors (For Knowledge)

Load Cell

Principle of Operation

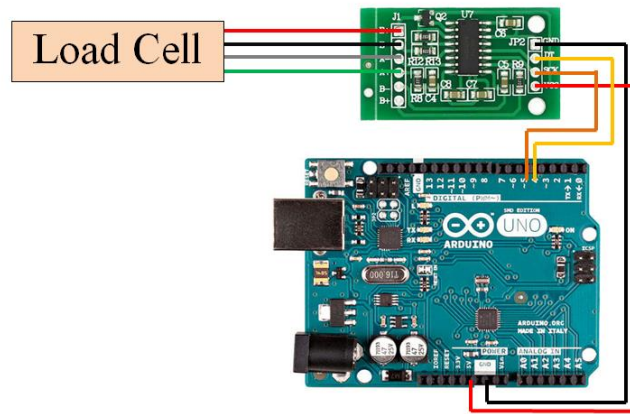
A load cell converts a force into an electrical signal that can be measured. The electrical signal changes proportionally to the force applied. Strain gauge load cells are composed of a metal bar with attached strain gauges (under the white glue). A strain gauge is an electrical sensor that measures force or strain on an object. The resistance of the strain gauges varies when an external force is applied to an object, which results in a deformation of the object's shape (in this case, the metal bar). The strain gauge resistance is proportional to the load applied, which allows us to calculate the weight of objects.



Hx711 Amplifier

Because the changes in strain when weighting objects are so small, we need an amplifier. This module uses 24 high-precision A / D converter. This chip is designed for high-precision electronic scale and design, has two analog input channels, programmable gain of 128 integrated amplifier.

Connection with Arduino



You can Find a more comprehensive guide [here](#)

CSE481/CSE 371 Spring 2026

Actuator with Arduino



Lab Objectives:

The objective of this lab is to understand the fundamental principles, structure, and operation of different types of DC motors. Students will learn:

1. DC Motor Control

- The working principle of DC motors, including brushed and brushless types.
- How to control the speed and direction of a DC motor using H-Bridge circuits (L298N) and Pulse Width Modulation (PWM) with an Arduino.

DC Motors

A DC motor (Direct Current motor) is an electromechanical device that converts electrical energy into mechanical energy using direct current (DC). It operates based on Faraday's Law of Electromagnetic Induction, which states that a current-carrying conductor placed in a magnetic field experiences a force that results in motion.

I. Working Principle

- When DC electricity flows through the motor's winding, it creates a magnetic field that interacts with the permanent magnets or electromagnets inside the motor. This interaction produces a force (Lorentz force) that causes the rotor (the rotating part) to spin. The direction of rotation is determined by Fleming's Left-Hand Rule.

II. Components of DC motor

A DC motor consists of several key components:

a) Stator (Stationary Part)

- The outer frame of the motor, which contains permanent magnets or field windings that generate a magnetic field.
- In some DC motors, the stator is an electromagnet, which requires an external power source.

b) Rotor (Rotating Part)

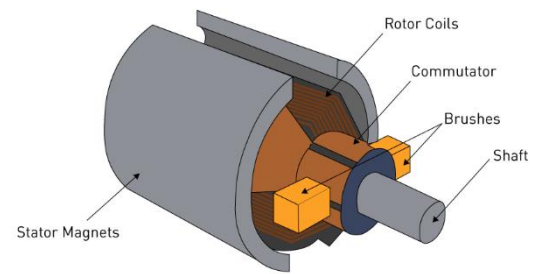
- The armature (a core made of iron laminations with copper windings) rotates inside the stator.
- When current flows through the armature windings, it experiences a force that causes it to turn.

c) Commutator

- A cylindrical switch made of copper segments connected to the armature.
- It ensures that the current direction in the armature winding reverses at the right time to maintain continuous rotation.

d) Brushes

- Carbon or graphite brushes press against the commutator.
- They transfer electricity from the external power source to the rotating armature windings.



III. Types of DC Motors

DC motors are classified based on how the field winding is connected to the armature winding:

a) Brushed DC Motors (Uses Commutator & Brushes)

1. Series DC Motor:

- Field winding is connected in series with the armature winding.
- Provides high torque but speed varies with load.
- Used in electric traction, cranes, and elevators.

2. Shunt DC Motor:

- Field winding is connected in parallel with the armature winding.
- Offers constant speed, even under load changes.
- Used in industrial machines and conveyors.

3. Compound DC Motor:

- Combination of series and shunt winding, offering both high torque and stable speed.
- Used in printing presses, rolling mills, and heavy-duty applications.

b) Brushless DC Motor (BLDC)

- No brushes or commutator; instead, electronic circuits control the current flow.
- More efficient, longer lifespan, and requires less maintenance.
- Used in drones, electric vehicles, and industrial automation.

IV. DC Motor Selection

1. Voltage

- Typical DC motors are rated from about **6V-12V**. The larger ones are often 24V or more.

2. Current

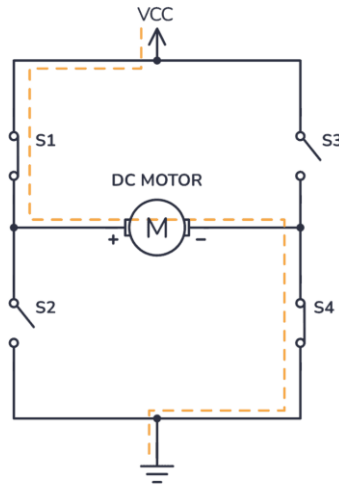
- **The operating current:** This is the average amount of current the motor is expected to draw under a typical torque.
- **The stall current:** This is when you power up the motor, but you put enough torque on it to force it to stop rotating.

3. Torque

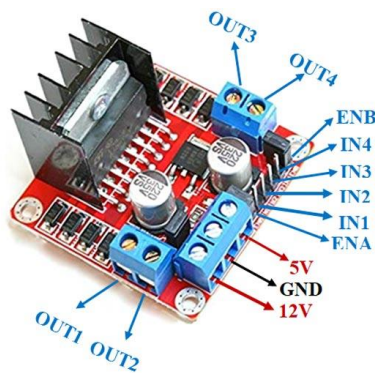
- **The operating torque:** This is the torque the motor was designed to give. Usually, it is the listed torque value.
- **The stall torque:** This is the torque required to stop the motor from rotating

H-Bridge

An H-bridge is an electronic circuit that allows a voltage to be applied across a load, such as a motor, in either direction. It's widely used in applications like controlling DC motors, enabling them to spin both clockwise and counterclockwise. The name "H-bridge" comes from its typical circuit diagram, which resembles the letter "H."



The **L298N** motor driver can be used to control two motors at the same time.



Pins Description:

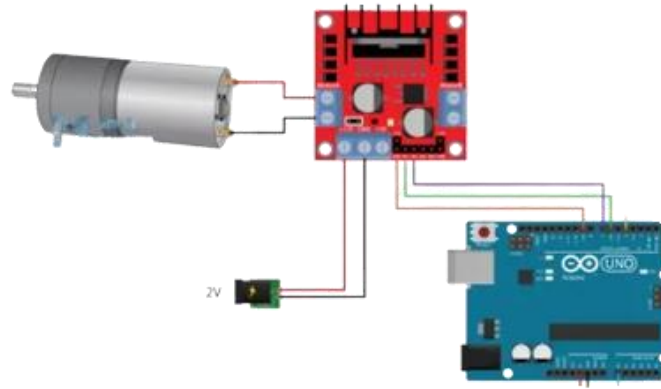
- **12V:** powers the internal IC of H-bridge which drives the motors.
- **5V:** powers the logic circuitry of IC.
- **GND:** common ground pin for IC.
- **ENA:** it is enable pin. It is used to enable motorA and control the speed of the motor. It is connected to an Arduino pin.
- **IN1, IN2:** controls the direction of motorA. It is connected to pins in the Arduino.
- **OUT1, OUT2:** output pins for motorA. It is connected to the motor.
- **ENB:** it is enable pin. It is used to enable motorB and control the speed of the motor. It is connected to an Arduino pin.
- **IN3, IN4:** controls the direction of motorB. It is connected to pins in the Arduino.
- **OUT3, OUT4:** output pins for motorB. It is connected to the motor.

It can control the motor directions using IN1 and IN2 pins:

IN1	IN2	Motor Status
GND	GND	Stopped
5V	GND	Turns in clockwise direction
GND	5V	Turns in counterclockwise direction
5V	5V	Stopped

Exercise 1

Try by yourself to connect the motor with L298N and turn the motor clockwise and counterclockwise



```
int enA = 9;
int IN1 = 5;
int IN2 = 6;
void setup() {
    // put your setup code here, to run once:
    pinMode(enA, OUTPUT);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
}

void loop() {
    // put your main code here, to run repeatedly:
    //turn the motor clockwise for 5 sec
    digitalWrite(enA, HIGH);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    delay(5000);
    //stop the motor for 1 sec
    digitalWrite(enA, LOW);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    delay(1000);
    //turn the motor counterclockwise for 3 sec
    digitalWrite(enA, HIGH);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
}
```

Pulse Width Modulation (PWM) for Speed Control

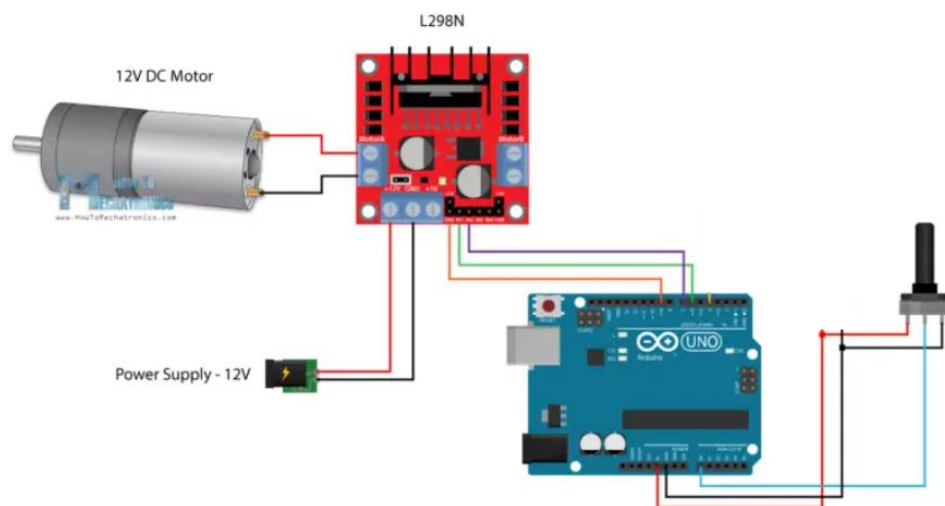
- Most Microcontrollers cannot generate analog output. However, this is tackled by a technique called PWM.
- Pulse Width Modulation, or PWM, is a technique for getting analog results with digital means.
- Digital control is used to create a square wave, a signal switched between on and off.
- This on-off pattern can simulate voltages in between full on (5 Volts) and off (0 Volts) by changing the portion of the time the signal spends on versus the time that the signal spends off.
- The duration of "on time" is called the pulse width.
- If you repeat this on-off pattern fast enough with an LED for example, the result is as if the signal is a steady voltage between 0 and 5v controlling the brightness of the LED.
- You can apply the same technique on motors to control its speed instead of using the full voltage. This can be done using PWM technique where you give the driver a signal where it represents the voltage between 0V and 5V.

$$duty_cycle = \frac{PulseWidth}{Period}$$
$$V_{required} = duty_cycle * V_{full}$$

- We can apply PWM with Arduino using the analog signal. Instead of giving the enable pin in the driver the full high, we will give it signal in between 0 to 255 (255 equals to 5V and 0 equals to 0V).

Exercise 2

Try by yourself to connect the motor with L298N and connect potentiometer with the Arduino to control the speed of the motor. Remap the reading from the potentiometer to be in between 0 to 255 (N.B. Arduino reads analog signal between 0 to 1023 but the output is between 0 to 255)




```
#define enA 9
#define in1 6
#define in2 7

void setup() {
  pinMode(enA, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  // Set initial rotation direction
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
}

void loop() {
  int potValue = analogRead(A0); // Read potentiometer value
  int pwmOutput = map(potValue, 0, 1023, 0, 255); // Map the potentiometer value from 0 to 255
  analogWrite(enA, pwmOutput); // Send PWM signal to L298N Enable pin
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
}
```